

```
!pip install -q pandas numpy scikit-learn pypdf python-docx sentence-transformers
```

```
import os
import re
import json
import numpy as np
import pandas as pd

from pypdf import PdfReader
from docx import Document

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

print("All libraries installed successfully.")
```

All libraries installed successfully.

```
SKILLS = [
    "Python",
    "Java",
    "C++",
    "JavaScript",
    "TypeScript",

    "SQL",
    "MySQL",
    "PostgreSQL",
    "MongoDB",
    "SQLite",

    "NumPy",
    "Pandas",
    "Matplotlib",
    "Seaborn",

    "Scikit-learn",
    "TensorFlow",
    "PyTorch",
```

"Keras",
"XGBoost",
"LightGBM",
"CatBoost",

"Machine Learning",
"Deep Learning",
"Artificial Intelligence",
"Natural Language Processing",
"Computer Vision",
"Reinforcement Learning",

"Feature Engineering",
"Data Preprocessing",
"Data Analysis",
"Statistics",
"Data Science",

"FastAPI",
"Flask",
"Django",
"REST API",
"API",

"Docker",
"Kubernetes",
"Git",
"GitHub",
"Linux",

"AWS",
"Google Cloud",
"Azure",
"GCP",

"Hugging Face",
"Transformers",
"BERT",
"LLM",
"Generative AI",

"OpenAI",
"LangChain",
"RAG",

```
"Spark",
"Apache Spark",

"HTML",
"CSS",
"React",
"Node.js"
]

print("Total skills:", len(SKILLS))
```

Total skills: 60

```
def clean_text(text):
    text = str(text)

    text = text.replace("\n", " ")
    text = text.replace("\r", " ")

    text = re.sub(r"\s+", " ", text)

    return text.strip()
```

```
sample = """
Python
Machine Learning

SQL
"""

print(clean_text(sample))
```

Python Machine Learning SQL

```
def extract_skills(text, skills=SKILLS):

    text_lower = text.lower()

    found = []
```

```
for skill in skills:

    pattern = r"(?<!\w)" + re.escape(skill.lower()) + r"(?!\\w)"

    if re.search(pattern, text_lower):
        found.append(skill)

return sorted(set(found))
```

```
test_text = """
I have experience with Python, Pandas, Scikit-learn,
SQL, Machine Learning and FastAPI.
"""
```

```
print(extract_skills(test_text))
```

```
['FastAPI', 'Machine Learning', 'Pandas', 'Python', 'SQL', 'Scikit-learn']
```

```
resume_text = """
Computer Science and Technology student interested in Machine Learning
and Artificial Intelligence.

Skills include Python, NumPy, Pandas, Matplotlib, Scikit-learn,
SQL, Machine Learning, Natural Language Processing, FastAPI,
Git, GitHub and Data Science.

Built machine learning projects involving classification,
regression, data preprocessing, feature engineering and model evaluation.
"""
```

```
job_text = """
We are looking for a Junior Machine Learning Engineer.

The candidate should have experience with Python, Machine Learning,
Scikit-learn, Pandas, NumPy and SQL.

Knowledge of Natural Language Processing, FastAPI, Docker,
AWS, Git, REST API and Deep Learning is preferred.

Responsibilities include developing machine learning models,
preprocessing datasets, feature engineering, model evaluation
```

```
and deploying machine learning applications.
"""
```

```
def calculate_tfidf_similarity(resume, job):

    vectorizer = TfidfVectorizer(
        stop_words="english",
        ngram_range=(1, 2)
    )

    matrix = vectorizer.fit_transform([
        resume,
        job
    ])

    similarity = cosine_similarity(
        matrix[0:1],
        matrix[1:2]
    )[0][0]

    return float(similarity * 100)
```

```
tfidf_score = calculate_tfidf_similarity(
    resume_text,
    job_text
)

print(f"TF-IDF Similarity: {tfidf_score:.2f}%")
```

TF-IDF Similarity: 37.90%

```
def calculate_skill_match(resume, job):

    resume_skills = set(extract_skills(resume))
    job_skills = set(extract_skills(job))

    matching = sorted(resume_skills & job_skills)

    missing = sorted(job_skills - resume_skills)

    extra = sorted(resume_skills - job_skills)
```

```

if len(job_skills) > 0:
    coverage = len(matching) / len(job_skills) * 100
else:
    coverage = 0

return {
    "resume_skills": sorted(resume_skills),
    "job_skills": sorted(job_skills),
    "matching_skills": matching,
    "missing_skills": missing,
    "extra_skills": extra,
    "skill_coverage": coverage
}

```

```

skill_result = calculate_skill_match(
    resume_text,
    job_text
)

print("Matching:", skill_result["matching_skills"])
print("Missing:", skill_result["missing_skills"])
print("Coverage:", skill_result["skill_coverage"])

```

Matching: ['FastAPI', 'Feature Engineering', 'Git', 'Machine Learning', 'Natural Language Processing', 'NumPy']
 Missing: ['API', 'AWS', 'Deep Learning', 'Docker', 'REST API']
 Coverage: 66.66666666666666

```

skill_result = calculate_skill_match(
    resume_text,
    job_text
)

print("Matching:", skill_result["matching_skills"])
print("Missing:", skill_result["missing_skills"])
print("Coverage:", skill_result["skill_coverage"])

```

Matching: ['FastAPI', 'Feature Engineering', 'Git', 'Machine Learning', 'Natural Language Processing', 'NumPy']
 Missing: ['API', 'AWS', 'Deep Learning', 'Docker', 'REST API']
 Coverage: 66.66666666666666

```
from sentence_transformers import SentenceTransformer

embedding_model = SentenceTransformer(
    "all-MiniLM-L6-v2"
)

print("Embedding model loaded.")
```

Loading weights: 100%

103/103 [00:00<00:00, 2791.65it/s]

Embedding model loaded.

```
def calculate_semantic_similarity(resume, job):

    embeddings = embedding_model.encode(
        [resume, job],
        normalize_embeddings=True
    )

    score = np.dot(
        embeddings[0],
        embeddings[1]
    )

    score = max(0, min(1, float(score)))

    return score * 100
```

```
semantic_score = calculate_semantic_similarity(
    resume_text,
    job_text
)

print(f"Semantic Similarity: {semantic_score:.2f}%")
```

Semantic Similarity: 73.38%

```
def calculate_final_score(resume, job):
```

```
tfidf = calculate_tfidf_similarity(
    resume,
    job
)

semantic = calculate_semantic_similarity(
    resume,
    job
)

skill_result = calculate_skill_match(
    resume,
    job
)

skill_coverage = skill_result["skill_coverage"]

final_score = (
    semantic * 0.50 +
    skill_coverage * 0.40 +
    tfidf * 0.10
)

return {
    "final_score": round(final_score, 2),
    "semantic_score": round(semantic, 2),
    "tfidf_score": round(tfidf, 2),
    "skill_coverage": round(skill_coverage, 2),
    **skill_result
}
```

```
result = calculate_final_score(
    resume_text,
    job_text
)

print("Final Match Score:", result["final_score"], "%")
print("Semantic Score:", result["semantic_score"], "%")
print("TF-IDF Score:", result["tfidf_score"], "%")
print("Skill Coverage:", result["skill_coverage"], "%")
```

Final Match Score: 67.14 %
Semantic Score: 73.38 %

TF-IDF Score: 37.9 %
Skill Coverage: 66.66666666666666 %

```
def generate_recommendations(result):  
    recommendations = []  
    missing = result["missing_skills"]  
    if missing:  
        for skill in missing:  
            recommendations.append(  
                f"Consider learning or improving {skill}."  
            )  
    if result["final_score"] < 50:  
        recommendations.append(  
            "Your resume has relatively low alignment with this job. "  
            "Consider tailoring your resume to the job requirements."  
        )  
    elif result["final_score"] < 75:  
        recommendations.append(  
            "Your profile has moderate alignment. "  
            "Adding missing technical skills could improve your match."  
        )  
    else:  
        recommendations.append(  
            "Your profile shows strong alignment with this position."  
        )  
    return recommendations
```

```
def analyze_resume(resume, job):  
    resume = clean_text(resume)  
    job = clean_text(job)
```

```
        result = calculate_final_score(
            resume,
            job
        )

        result["recommendations"] = generate_recommendations(
            result
        )

    return result
```

```
result = analyze_resume(
    resume_text,
    job_text
)
```

```
print("=" * 50)
print("          AI RESUME-JOB MATCHER")
print("=" * 50)

print(f"\nFinal Match Score: {result['final_score']}%")

print(f"Semantic Similarity: {result['semantic_score']}%")

print(f"TF-IDF Similarity: {result['tfidf_score']}%")

print(f"Skill Coverage: {result['skill_coverage']}%")

print("\nMATCHING SKILLS")

for skill in result["matching_skills"]:
    print("✓", skill)

print("\nMISSING SKILLS")

for skill in result["missing_skills"]:
    print("✗", skill)

print("\nRECOMMENDATIONS")

for recommendation in result["recommendations"]:
    print("→", recommendation)
```

```
=====
AI RESUME-JOB MATCHER
=====
```

```
Final Match Score: 67.14%
Semantic Similarity: 73.38%
TF-IDF Similarity: 37.9%
Skill Coverage: 66.66666666666666%
```

```
MATCHING SKILLS
```

```
✓ FastAPI
✓ Feature Engineering
✓ Git
✓ Machine Learning
✓ Natural Language Processing
✓ NumPy
✓ Pandas
✓ Python
✓ SQL
✓ Scikit-learn
```

```
MISSING SKILLS
```

```
✗ API
✗ AWS
✗ Deep Learning
✗ Docker
✗ REST API
```

```
RECOMMENDATIONS
```

```
→ Consider learning or improving API.
→ Consider learning or improving AWS.
→ Consider learning or improving Deep Learning.
→ Consider learning or improving Docker.
→ Consider learning or improving REST API.
→ Your profile has moderate alignment. Adding missing technical skills could improve your match.
```

```
def extract_pdf_text(file_path):

    reader = PdfReader(file_path)
```

```
pages = []

for page in reader.pages:

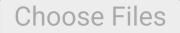
    text = page.extract_text()

    if text:
        pages.append(text)

return "\n".join(pages)
```

```
from google.colab import files

uploaded = files.upload()
```

 data.pdf
data.pdf(application/pdf) - 304677 bytes, last modified: 7/12/2026 - 100% done
Saving data.pdf to data (1).pdf

```
resume_from_pdf = extract_pdf_text(
    "data.pdf"
)

print(resume_from_pdf[:3000])
```

```
def extract_docx_text(file_path):

    document = Document(file_path)

    paragraphs = []

    for paragraph in document.paragraphs:

        if paragraph.text.strip():
            paragraphs.append(paragraph.text)
```

```
return "\n".join(paragraphs)
```

```
%%writefile app.py

from fastapi import FastAPI, UploadFile, File, Form
from fastapi.responses import HTMLResponse
from pypdf import PdfReader
from sentence_transformers import SentenceTransformer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

import re
import numpy as np
import io

app = FastAPI(
    title="AI Resume Job Matcher"
)

SKILLS = [
    "Python", "Java", "C++", "JavaScript",
    "SQL", "MySQL", "PostgreSQL", "MongoDB",
    "NumPy", "Pandas", "Matplotlib", "Seaborn",
    "Scikit-learn", "TensorFlow", "PyTorch",
    "Keras", "XGBoost", "LightGBM", "CatBoost",
    "Machine Learning", "Deep Learning",
    "Artificial Intelligence",
    "Natural Language Processing",
    "Computer Vision",
    "Feature Engineering",
    "Data Preprocessing",
    "Data Analysis",
    "Statistics", "Data Science",
    "FastAPI", "Flask", "Django",
    "REST API", "Docker", "Kubernetes",
    "Git", "GitHub", "Linux",
    "AWS", "Google Cloud", "Azure", "GCP",
    "Hugging Face", "Transformers", "BERT",
    "LLM", "Generative AI", "OpenAI",
    "LangChain", "RAG",
```

```

    "Apache Spark", "React", "Node.js"
]

model = SentenceTransformer(
    "all-MiniLM-L6-v2"
)

def clean_text(text):
    text = str(text)

    text = text.replace("\n", " ")
    text = text.replace("\r", " ")

    text = re.sub(r"\s+", " ", text)

    return text.strip()

def extract_skills(text):
    text_lower = text.lower()

    found = []

    for skill in SKILLS:
        pattern = (
            r"(?<!\w)"
            + re.escape(skill.lower())
            + r"(?!\\w)"
        )

        if re.search(pattern, text_lower):
            found.append(skill)

    return sorted(set(found))

def extract_pdf_text(data):
    reader = PdfReader(

```

```

        io.BytesIO(data)
    )

    pages = []

    for page in reader.pages:
        text = page.extract_text()

        if text:
            pages.append(text)

    return "\n".join(pages)

def tfidf_score(resume, job):
    vectorizer = TfidfVectorizer(
        stop_words="english",
        ngram_range=(1, 2)
    )

    matrix = vectorizer.fit_transform(
        [resume, job]
    )

    score = cosine_similarity(
        matrix[0:1],
        matrix[1:2]
    )[0][0]

    return float(score * 100)

def semantic_score(resume, job):
    embeddings = model.encode(
        [resume, job],
        normalize_embeddings=True
    )

    score = np.dot(
        embeddings[0],
        embeddings[1]
    )

```

```
)

return float(
    max(0, min(1, score)) * 100
)

def analyze(resume, job):

    resume = clean_text(resume)
    job = clean_text(job)

    resume_skills = set(
        extract_skills(resume)
    )

    job_skills = set(
        extract_skills(job)
    )

    matching = sorted(
        resume_skills & job_skills
    )

    missing = sorted(
        job_skills - resume_skills
    )

    coverage = (
        len(matching) / len(job_skills) * 100
        if job_skills else 0
    )

    tfidf = tfidf_score(
        resume,
        job
    )

    semantic = semantic_score(
        resume,
        job
    )

    final = (
```



```
        semantic * 0.50 +
        coverage * 0.40 +
        tfidf * 0.10
    )

    return {
        "final": round(final, 2),
        "semantic": round(semantic, 2),
        "tfidf": round(tfidf, 2),
        "coverage": round(coverage, 2),
        "matching": matching,
        "missing": missing
    }
}
```

```
HTML = ""
<!DOCTYPE html>

<html>

<head>

<meta name="viewport"
content="width=device-width, initial-scale=1">

<title>AI Resume Job Matcher</title>

<style>

* {
    box-sizing: border-box;
}

body {
    margin: 0;
    font-family: Arial, sans-serif;
    background: #0b1020;
    color: white;
}

.container {
    width: min(1000px, 94%);
    margin: auto;
    padding: 30px 0;
```

```
}

.hero {
  text-align: center;
  margin-bottom: 30px;
}

.hero h1 {
  font-size: clamp(30px, 7vw, 55px);
  margin-bottom: 10px;
}

.hero p {
  color: #aab3c5;
}

.card {
  background: #121a2e;
  border: 1px solid #293551;
  border-radius: 18px;
  padding: 22px;
  margin-bottom: 20px;
}

label {
  display: block;
  margin-bottom: 8px;
  font-weight: bold;
}

textarea {
  width: 100%;
  min-height: 220px;
  resize: vertical;
  border: 1px solid #3b4865;
  background: #0c1325;
  color: white;
  border-radius: 12px;
  padding: 15px;
  outline: none;
}

textarea:focus {
  border-color: #7c8cff;
```

```
}

input[type=file] {
  width: 100%;
  padding: 15px;
  border: 1px dashed #52617e;
  border-radius: 12px;
  margin-bottom: 15px;
}

button {
  width: 100%;
  padding: 15px;
  border-radius: 12px;
  border: 1px solid #8b96ff;
  background: transparent;
  color: white;
  font-size: 16px;
  cursor: pointer;
  transition: .2s;
}

button:hover {
  background: #7c8cff;
  color: #080b16;
  transform: translateY(-2px);
}

.score {
  text-align: center;
  font-size: 60px;
  font-weight: bold;
  margin: 20px;
}

.grid {
  display: grid;
  grid-template-columns:
    repeat(auto-fit, minmax(220px, 1fr));
  gap: 15px;
}

.metric {
  padding: 20px;
```

```
        border: 1px solid #303d5a;
        border-radius: 14px;
        text-align: center;
    }

    .metric strong {
        display: block;
        font-size: 28px;
        margin-top: 8px;
    }

    .skill {
        display: inline-block;
        border: 1px solid #506080;
        padding: 7px 10px;
        border-radius: 20px;
        margin: 4px;
        font-size: 14px;
    }

    .missing {
        border-color: #a96d6d;
    }

    .matching {
        border-color: #6b9d7c;
    }

    .hidden {
        display: none;
    }

    .error {
        color: #ff8f8f;
    }

</style>

</head>

<body>

<div class="container">
```

```
<div class="hero">

<h1>AI Resume-Job Matcher</h1>

<p>
NLP-powered resume analysis and skill-gap detection
</p>

</div>
```

```
<div class="card">

<form
action="/analyze"
method="post"
enctype="multipart/form-data">

<label>
Upload Resume
</label>

<input
type="file"
name="resume_file"
accept=".pdf">

<label>
Or paste your resume
</label>

<textarea
name="resume_text"
placeholder="Paste your resume here...">
</textarea>

<label>
Job Description
</label>

<textarea
```

```
name="job_description"
placeholder="Paste the job description here..."
required>
</textarea>
```

```
<br><br>
```

```
<button type="submit">
Analyze Resume
</button>
```

```
</form>
```

```
</div>
```

```
</div>
```

```
</body>
```

```
</html>
"""
```

```
@app.get("/", response_class=HTMLResponse)
def home():
```

```
    return HTML
```

```
@app.post(
    "/analyze",
    response_class=HTMLResponse
)
```

```
async def analyze_resume(
```

```
    resume_file: UploadFile = File(None),
```

```
    resume_text: str = Form(""),
```

```
    job_description: str = Form(...)
):
```

```
    if resume_file:
```

```

    data = await resume_file.read()

    if resume_file.filename.lower().endswith(".pdf"):

        resume_text = extract_pdf_text(data)

if not resume_text.strip():

    return HTMLResponse(
        "<h2>Please provide a resume.</h2>"
    )

result = analyze(
    resume_text,
    job_description
)

matching_html = "".join(
    f'<span class="skill matching">✓ {x}</span>'
    for x in result["matching"]
)

missing_html = "".join(
    f'<span class="skill missing">✗ {x}</span>'
    for x in result["missing"]
)

page = f"""

<!DOCTYPE html>

<html>

<head>

<meta name="viewport"
content="width=device-width, initial-scale=1">

<title>Analysis Result</title>

<style>

body {{

```

```
    margin: 0;
    background: #0b1020;
    color: white;
    font-family: Arial;
  }}

  .container {{
    width: min(950px, 94%);
    margin: auto;
    padding: 30px 0;
  }}

  .card {{
    background: #121a2e;
    border: 1px solid #293551;
    border-radius: 18px;
    padding: 25px;
    margin-bottom: 20px;
  }}

  .score {{
    text-align: center;
    font-size: 70px;
    font-weight: bold;
  }}

  .grid {{
    display: grid;
    grid-template-columns:
      repeat(auto-fit, minmax(180px, 1fr));
    gap: 15px;
  }}

  .metric {{
    text-align: center;
    padding: 20px;
    border: 1px solid #303d5a;
    border-radius: 14px;
  }}

  .metric strong {{
    display: block;
    font-size: 28px;
    margin-top: 8px;
```



```
}}

.skill {{
    display: inline-block;
    padding: 8px 12px;
    border: 1px solid #52617e;
    border-radius: 20px;
    margin: 4px;
}}

.back {{
    display: block;
    text-align: center;
    margin-top: 25px;
    color: white;
    text-decoration: none;
    border: 1px solid #7c8cff;
    padding: 13px;
    border-radius: 12px;
}}

</style>

</head>

<body>

<div class="container">

<div class="card">

<h1>Resume Analysis</h1>

<div class="score">
{result["final"]}
</div>

<p style="text-align:center">
Overall Match Score
</p>

</div>
```

```
<div class="card">

<h2>Model Metrics</h2>

<div class="grid">

<div class="metric">
TF-IDF
<strong>{result["tfidf"]}%%</strong>
</div>

<div class="metric">
Semantic
<strong>{result["semantic"]}%%</strong>
</div>

<div class="metric">
Skill Coverage
<strong>{result["coverage"]}%%</strong>
</div>

</div>

</div>
```

```
<div class="card">

<h2>Matching Skills</h2>

{matching_html or "No matching skills detected."}

</div>
```

```
<div class="card">

<h2>Missing Skills</h2>

{missing_html or "No major missing skills detected."}

</div>
```

```
<a class="back" href="/">
Analyze Another Resume
</a>

</div>

</body>

</html>
"""

return page
```

Overwriting app.py

```
%%writefile requirements.txt

fastapi
uvicorn
python-multipart
numpy
scikit-learn
pypdf
sentence-transformers
torch
```

Overwriting requirements.txt

```
!pip install -q fastapi uvicorn python-multipart
```

```
!uvicorn app:app --host 0.0.0.0 --port 8000 &
```

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate l
Loading weights: 100% 103/103 [00:00<00:00, 12044.87it/s]
INFO:      Started server process [33938]
INFO:      Waiting for application startup.
INFO:      Application startup complete.
ERROR:     [Errno 98] error while attempting to bind on address ('0.0.0.0', 8000): address already in use
INFO:      Waiting for application shutdown.
INFO:      Application shutdown complete.
```

```
!pip install -q pyngrok
```

```
!fuser -k 8000/tcp
```

```
8000/tcp:          33095
```

```
!lsof -i :8000
```

```
!uvicorn app:app --host 0.0.0.0 --port 8000 > server.log 2>&1 &
```

```
!cat server.log
```

```
from google.colab import files  
  
files.download("app.py")  
files.download("requirements.txt")
```